

VQ-LLM: High-performance Code Generation for Vector Quantization Augmented LLM Inference

Zihan Liu^{1,2}, Xinhao Luo^{1,2}, Junxian Guo¹, Wentao Ni¹, Yangjie Zhou¹, Yue Guan^{1,2}, Cong Guo³, Weihao Cui^{1,4}, Yu Feng¹, Minyi Guo^{1,2}, Yuhao Zhu⁵, Minjia Zhang⁶, Chen Jin⁷, Jingwen Leng^{1,2,*}

¹Shanghai Jiao Tong University, ²Shanghai Qi Zhi Institute, ³Duke University, ⁴National University of Singapore

⁵University of Rochester, ⁶University of Illinois Urbana-Champaign, ⁷Magik Compute

{altair.liu, lxx666, guojunxian, wennitao, yj_zhou, bonboru, weihao, y-feng}@sjtu.edu.cn, cong.guo@duke.edu
guo-my@cs.sjtu.edu.cn, yzhu@rochester.edu, minjiaz@illinois.edu, chenjin@magikcompute.ai, leng-jw@cs.sjtu.edu.cn

Abstract—Vector quantization (VQ), which treats a vector as a compression unit, gains increasing research interests for its potential to accelerate large language models (LLMs). Compared to conventional element-wise quantization methods, VQ algorithms can compress weight and KV cache tensors in LLMs with a greater ratio while maintaining the high model accuracy. However, translating a VQ algorithm’s memory reduction into the actual latency improvement is challenging. We profile and analyze the current approach of integrating VQ into computation kernels and show that its major inefficiency lies in the poor access efficiency of codebooks in VQ algorithms and uncoordinated computation dataflow. Meanwhile, the diversity of VQ algorithms (e.g., different vector sizes and entry counts) and LLMs’ computation kernels (e.g matrix-matrix/vector multiplication and attention computation) makes it impractical to manually craft efficient kernel implementations for each specific case.

In this work, we design and implement VQ-LLM, an efficient fused VQ kernel generation framework. We first introduce a software abstraction called codebook cache to optimize codebook access efficiency and support the integration of VQ with various computations. The codebook cache adaptively stores different entries across the GPU’s memory hierarchy, including off-chip global memory, on-chip shared memory, and registers. Centered around the codebook cache, we design an efficient computation engine that optimizes memory traffic during computations involving codebooks. This compute engine adopts the codebook-centric dataflow and fusion optimizations. Additionally, we provide adaptive heuristics to tailor parameter selection in our optimizations to diverse VQ configurations. Our optimizations achieve the latency reduction of 64.36% to 99.1% compared to existing open-source implementations. A final comparison with state-of-the-art element-wise quantization methods like AWQ and QoQ shows that our VQ-LLM is practically viable, achieving latencies close or even better latencies to those at equivalent bit-widths, potentially offering greater accuracy.

I. INTRODUCTION

With the great success of large language models (LLMs), neural networks are placing significant pressure on current hardware, especially memory systems [21], [27], [28], [33], [74], [75]. Quantization techniques become essential for deploying these large models [15], [18]–[20], [24], [30], [51], [62]. Quantization reduces the original IEEE-754 half format FP16 data to types with much narrower bit-widths, such as FP8 and INT4, decreasing the memory footprint significantly [1]. Researchers have developed numerous novel data

formats and algorithms, like MXFP and ANT, with varying scaling granularities to represent the original data using fewer bits [20], [49]. However, these techniques treat each data point as an independent element for compression, overlooking the potential information between elements. As a result, these methods typically reach a 4-bit limit; compressing to 2 bits or less leads to a substantial accuracy loss [12], [15], [56].

Under these scenarios, vector quantization (VQ) emerges as a pivotal technique to further reduce LLMs’ memory footprints [12], [56], [57], [67], [69]. The VQ methods compress a vector of multiple elements into a single element and enabling the capture of information across elements [57], [69]. Typically, this cross-element information is gathered through clustering, which involves applying a clustering algorithm to all vectors and using cluster centroids to represent nearby vectors [26], [37]. Furthermore, some researchers suggest iteratively processing the residuals between the original and quantized data to enhance reconstruction quality [32], [63]. For LLMs, VQ achieves higher accuracy at the same 4-bit level or maintains equivalent accuracy at 2-bits, and some approaches can compress the KV cache in LLMs to 1-bit [69].

Despite their appealing accuracy and compression ratios, VQ-augmented LLMs do not significantly enhance the model’s latency performance in practice. Our analysis in Sec. III indicates that existing VQ methods have substantially higher latency than conventional element-wise quantization methods, often performing worse than the original FP16 version. The inefficiencies primarily stem from how memory access and computation dataflow are managed when interacting with the codebooks in VQ methods. We have identified three key challenges that must be addressed to generate high-performance kernels integrating VQ with subsequent computations.

The first challenge lies in the placement of VQ’s codebooks. We find that the common practice of storing all codebook entries in GPU shared memory increases shared memory usage, thereby reducing the number of thread blocks that can concurrently operate on each SM, which diminishes performance. Additionally, the number of codebook entries far exceeds the number of available shared memory banks, leading to significant bank conflicts. The second challenge involves coordinating the loading of codebooks and subsequent computation. There is excessive traffic in loading the codebook

*Jingwen Leng is the corresponding author of this paper.

from global memory to shared memory, and in transferring codebook entries from shared memory to registers, which should be much lower in theory. The reasons include multiple thread blocks loading duplicate codebooks, and the requirement for threads to store data reconstructed via codebook entries (we refer to them as dequantized data throughout the paper) back to shared memory in a layout that differs from their dequantization for subsequent computations. The last challenge is that the diversity of VQ algorithms (e.g., different vector sizes and entry counts) and LLMs' computation kernels (e.g. matrix-matrix/vector multiplication and attention computation) makes it impractical to manually craft efficient kernel implementations for each specific case.

To address the challenges, this work develops VQ-LLM, an automatic high-performance fused VQ kernel code generation framework. We begin by introducing a software abstraction called **codebook cache**, designed to optimize codebook access efficiency and facilitate the integration of VQ with various computations. This cache enables efficient codebook placement across the GPU's memory hierarchy. We have identified that only a select few entries are accessed frequently. Therefore, rather than indiscriminately caching all entries in shared memory, we adopt a hierarchical approach: entries with low access frequency remain in global memory, while those accessed more frequently are cached in shared memory. To address inevitable bank conflicts, entries that are accessed extremely frequently are stored in thread-local registers, eliminating bank conflict issues. Furthermore, to mitigate negative impacts on computation (reduced concurrency), we utilize available slacks, which ensures no drop in resource utilization, to adaptively determine the optimal placement of entries.

Centering the codebook cache, we design an efficient compute engine that optimizes memory traffic when computing with codebooks, and it consists of two novel techniques. The first called codebook-centric dataflow divides and parallelizes the original computation task in a way that minimizes the codebook switch overhead. It may split the reduction dimension of the original computation task, for which we adaptively determine the split factor to balance the global reduction. The second technique, codebook-centric hierarchical fusion, extends the default shared memory level fusion to support the additional register-level fusion. This mechanism leverages a GPU feature known as intra-warp data exchange [42] to rearrange the dequantized data into the required layout for subsequent computations directly in registers. We adaptively decide where to conduct the fusion based on profiled exchanging overhead and difference between layout of dequantized data and layout required by subsequent computation.

Our evaluation shows that VQ-LLM achieves the latency reduction of 64.36% to 99.1% compared to compared to existing open-source implementations [13], [56]. We also perform extensive sensitivity to verify the effectiveness of each technique in our framework. A final comparison with state-of-the-art element-wise quantization methods like AWQ [30] and QoQ [31] shows that our VQ-LLM is practically viable, achieving latencies close or even better latencies to those at

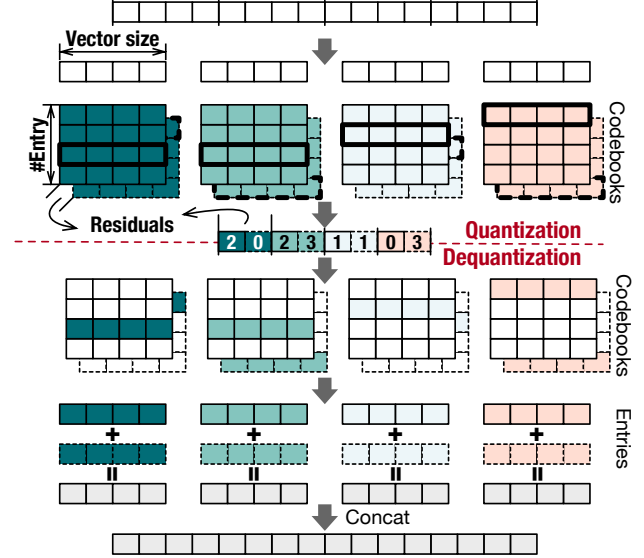


Fig. 1. Typical vector quantization pipeline.

equivalent bit-widths, potentially offering greater accuracy.

We list our main contributions as follow:

- To the best of our knowledge, we are the first to deeply dive into performance issues of vector quantization and make it practically feasible in LLM inference.
- We deliver a detailed analysis and identify these issues are caused by inefficient codebook entries access and uncoordinated codebook loading and subsequent computation.
- Based on the finding, we propose VQ-LLM to generate efficient fused VQ kernel implementation, it consists of codebook cache and codebook based compute engine, with configurable parameters and adaptive heuristics.
- We compare VQ-LLM with open-sources implementations and element-wise quantization works, with detail speed-up breakdown analysis on proposed optimizations.

II. BACKGROUND AND RELATED WORKS

This section first introduces the basic concept of vector quantization and its applications in quantizing large language models. It then provides a detailed analysis of serving vector quantized large language models with existing solutions.

A. Vector Quantization (VQ)

Compared to traditional quantization, vector quantization (VQ) treats the vector of multiple elements as a unit and uses trained quantization points organized into codebooks to quantize the vector into a single element, rather than in an element-wise manner as in traditional quantization. This technique is widely used in vector database, nearest neighbor search, etc. [29], [34] VQ has several configurable parameters, highlighted in Fig. 1, which allow it to be specified for product quantization (PQ), additive quantization (AQ), and hybrid quantization (PRQ) [4], [11], [17], [25]. Apart from

these, there are other techniques such as hash-based [23] and lattice-based methods [2]. However, these techniques either cannot reconstruct the original data or need to be used in conjunction with PQ, AQ, and PRQ. Therefore, we do not delve into these techniques as they do not influence the core findings and insights of this work.

Typical VQ Pipeline. We use the example in Fig. 1 to demonstrate the typical VQ pipeline, and numbers in (·) represent the value of parameters in this example. We also summarize the VQ parameters in Tbl. I. First, the original 16-dimensional vectors are split into four sets of **vector size** (4)-dimensional sub-vectors. Next, we collect sub-vectors in one sub-space (or several sub-spaces, depending on algorithms) and conduct k-means clustering to group these sub-vectors into **#Entry** (4) clusters. The original sub-vectors are then replaced with the index of their closest cluster centroids, using $\log_2 \text{#Entry}$ (2) bits. Next, we collect the differences between the original sub-vectors and their closest cluster centroids as the residuals. We then perform another round of k-means clustering and replace the residual sub-vectors with the index of the closest centroids of the new clusters. This process of residual quantization can be repeated, as determined by the **Residual** (2) parameter. The quantization process is now complete, as shown in the upper part of Fig. 1. We then gather all the aforementioned cluster centroids and organize them into codebooks. In the following sections, we refer to these centroids as codebook entries.

To reconstruct the original data, a dequantization process is required, as shown in the lower part of Fig. 1. For each residual, we use its quantized data to look up the corresponding codebooks and find the codebook entry indexed by the quantized data in each sub-space. We then gather the results from the same sub-spaces across different residuals, typically via element-wise accumulation. Finally, we concatenate the results from all sub-spaces. Throughout the entire process, **vector size**, **#Entry**, and **Residual** are configurable. These configurations are annotated with $\mathbf{x}, \mathbf{y}, \mathbf{z}$, in the format of $\text{VQ} \langle \mathbf{x}, \mathbf{y}, \mathbf{z} \rangle$. In this example, the configuration is $\text{VQ} \langle 4, 2, 2 \rangle$.

B. Large Language Models (LLMs)

LLMs adopt the Transformer architecture [58], which is pivotal in processing and generating natural language in sequences of tokens. The core of the Transformer architecture is multi-head attention (MHA), designed to run several parallel attention processes, allowing the model to simultaneously focus on different types of information from a single input sequence.

TABLE I
PARAMETERS OF VQ ALGORITHMS

Item	Description	Value in Sec. III
Vector size	Number of elements to quantize at once	4
#Entry	Number of quantization points (entries)	2^8
Residual	Number of times to quantize residual data	1

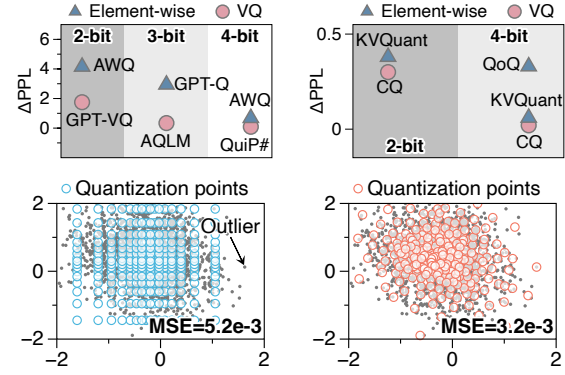


Fig. 2. (Upper) Accuracy of VQ and element-wise quantization, left is weight and right is KV cache quantization. (Lower) VQ (right) can better capture the distribution of data than element-wise quantization (left), with inter-dimensions information.

Each head in MHA can be thought of as an independent attention layer with its own learnable parameters. Outputs of these heads are then concatenated and fed to subsequent operations. Mathematically, MHA can be described as follows:

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \\ \text{head}_i &= \text{Attention}(Q = HW_i^Q, K = HW_i^K, V = HW_i^V), \\ \text{Attention}(Q, K, V) &= \text{softmax}(QK^T / \sqrt{d_k})V. \end{aligned}$$

Here, W_i^Q , W_i^K , W_i^V , and W^O are parameter matrices for the i -th head and the output projection, respectively. And H is the hidden state. The *softmax* function is applied over the keys to normalize their weights, ensuring that the output is a weighted sum of the values based on the input's relevance.

In the context of text generation, LLMs often first implement a prefill stage where the model processes existing tokens before generating new ones. This sets the initial state of the model's memory and attention mechanisms, making the generation process more context-aware. Following this, the decode phase begins, during which the model generates one token at a time, updating its internal state based on both the newly generated token and the preceding context. To efficiently reuse previously computed token representations during the decode phase, a Key-Value (KV) cache mechanism is often utilized [46], [68], enhancing inference performance.

C. VQ for LLM Acceleration

VQ gains increasing interests for its great potential for compressing and accelerating LLMs. This is because LLMs are highly memory-bound [61], with many researchers identifying weights and KV-cache as the main bottlenecks, accounting for over 95% of the memory footprint [28]. To further compress the weights and KV-cache and reduce memory usage, VQ has come to the center of the stage with its superior compression ratio and reconstruction quality. Various newly proposed VQ-based compression algorithms outperform SOTA element-wise quantization baselines in both weight-only compression (AWQ [30]) and KV-cache compression (KVQuant [24], QoQ [31]) under the same equivalent bitwidth [12], [56],

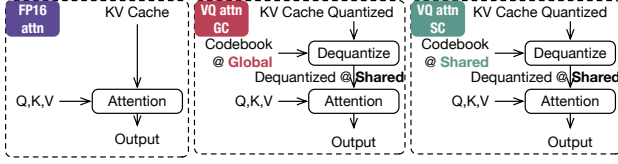


Fig. 3. Workflow of investigated VQ kernels.

[57], [67], [69], as shown in the upper part of Fig. 2. Some can even achieve higher quality with fewer equivalent bits. The underlying reason is depicted in the lower part of Fig. 2. With cross-dimension information, VQ can better capture the distribution characteristics of the data, resulting in lower reconstruction error. In contrast, traditional quantization relies on the Cartesian product of quantization points between dimensions and cannot represent some outliers well.

While converting the reduced memory footprint to actual speed-up is challenging due to the need for efficient kernels that take quantized data and codebooks as inputs, dequantize them, and perform computations. Unfortunately, existing algorithms only provide kernels with high latency, making them impractical for use [12], [56], as verified in Sec. VII. In the VQ pipeline, dequantization is the main bottleneck in the context of LLMs. This is because quantization can be done offline (for weights) or asynchronously with tiny overhead (for KV cache, also discussed in Sec. VII). However, dequantization is required every time before a computation since the quantized data store codebook indices and cannot be directly operated on. Therefore, this paper focuses on developing efficient fused dequantization-computation kernels.

In the next section, we will analyze the inefficiencies of existing and vanilla optimized fused dequantization-computation kernel. As mentioned before, the core difference between VQ and element-wise quantization is the use of vectorized codebooks, and we primarily focus on them in our analysis.

Noted that we target NVIDIA GPUs in this paper, although GPUs from other vendors share similar concepts [3], [39], [40], [54]. A GPU compute kernel launches thousands of threads, organized into thread blocks within a grid. Each thread block is dispatched to a Streaming Multiprocessor (SM), which may handle multiple thread blocks to overlap instructions [70]. Threads access three memory hierarchies: registers (local to each thread), shared memory (local to the thread block), and global memory (accessible by all threads).

III. MOTIVATION

In this section, we analyze the inefficiencies of current VQ implementation centering how codebooks are placed and utilized. We first outline our setup for a micro-benchmark-based investigation in Fig. 3 and then analyze it in detail.

A. Investigation Setup

We evaluate an attention kernel from Llama-7B [55] with 32 heads and head dimension of 128 on an RTX 4090 GPU [44]. We investigate three implementations of vector quantized (VQ) KV cache with the configuration $\mathbf{VQ}<4, 8, 1>$ that follows

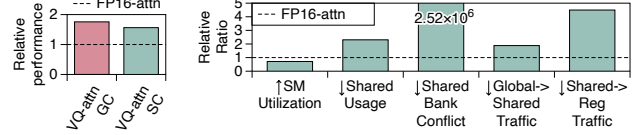


Fig. 4. (left) Latency of **VQ-attn-GC** and **VQ-attn-SC** relative to **FP16-attn**. (right) Relative performance counters of **VQ-attn-SC**.

CQ-2 [69]. As illustrated in Fig. 3, the first **FP16-attn** version implements Flash Decoding [10] from the FlashAttention library [7], [9]. We implement the **VQ-attn-GC** version ourselves following the original paper [12], [56], [57], [69] due to the lack of open-source kernels. **VQ-attn-GC** receives the VQ quantized KV cache and its codebooks, dequantizes them to FP16 precision, and performs the subsequent attention computation, with codebooks stored in *global memory*. Given the long access latency of global memory, we propose and implement another optimized version that stores codebooks in *shared memory* and hence is labelled as **VQ-attn-SC**, with the rest of the process mirroring that of **VQ-attn-GC**. Here we only analyze attention kernel thus KV cache compression, while these observations can also be generalized to GeMM/GeMV and weight compression.

B. Inefficiency Analysis

Since the attention (decoding) process is highly memory-bound, using $\mathbf{VQ}<4, 8, 1>$, which compresses the KV cache to 1/8, should significantly enhance its performance. However, as depicted on the left of Fig. 4, both VQ versions underperform the FP16 baseline. We also observe that the shared-memory-based codebook version, **VQ-attn-SC**, outperforms the global-memory-based version, **VQ-attn-GC**, demonstrating the effectiveness of utilizing shared memory for codebooks. Although shared memory and the GPU L1 cache share the same physical space, the hardware-managed L1 cache fails to capture the temporal locality of codebook entries. This is because the size and irregular access pattern of the entries does not align with the cache line size and prefetch width (128 bytes [41]) of the L1 cache. According to our profiling results, **VQ-attn-GC** achieves only a 12.45% L1 cache hit rate, indicating significant wasted capacity in the L1 cache. Consequently, we default to the **VQ-attn-SC** version to investigate its sources of inefficiencies.

Inefficient Codebook Access. Fig. 4 (right) compares the various performance counters of the **VQ-attn-SC** version and the FP16 version. We first observe an over 30% drop in compute (SM) utilization in the **VQ-attn-SC** version (1st bar). This decline is attributed to the VQ's significantly increasing shared memory footprint (2nd bar), which reduces the number of thread blocks that can run concurrently on each SM, leading to decreased performance. Additionally, we note high bank conflicts (3rd bar), indicative of highly serialized access to shared memory. Eliminating these bank conflicts is challenging for several reasons. First, the number of codebook entries vastly exceeds the number of shared memory

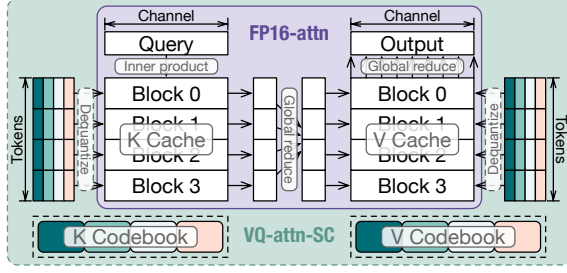


Fig. 5. Dataflow of **FP16-attn** (inner box) and **VQ-attn-SC** (outer box).

banks, e.g., 256 entries versus 32 banks, and their accesses are random during the VQ dequantization process, precluding the use of common static reordering or padding solutions for coalesced accesses [41]. It is possible to reorder entries or threads at runtime, which can introduce extra complexity and overhead. Second, a single codebook entry can occupy multiple banks in VQ, exacerbating the difficulty of mitigating bank conflicts.

Takeaway 1 Storing codebooks in fast on-chip buffers like shared memory is necessary, but not trivial.

Uncoordinated Codebook Load and Compute. The 4th bar in Fig. 4 (right) indicates that the traffic from off-chip global to on-chip shared memory is higher for the VQ version than for the FP16 version. This is counterintuitive since VQ is expected to significantly reduce global memory access. The cause of this unexpected off-chip traffic is that integrating VQ into the original compute kernel results in uncoordinated and duplicated loads of codebooks.

The inner box of Fig. 5 shows the original FlashDecoding’s dataflow [10], which parallelizes the computation of different tokens and computes the local softmax in global memory. When integrating the VQ codebooks to this computation dataflow, computing every four channels for a token needs to switch to a different codebook, following the VQ algorithm of CQ-2 [69]. Consequently, thread blocks handling different tokens end up accessing and loading identical codebooks as they process data across all channels, as shown in the outer box of Fig. 5. This results in significant duplicated off-chip memory traffic, and this challenge is also presented in the integration of VQ with GeMM kernels. For GPTVQ-2 [57],

TABLE II
VQ ALGORITHM AND THEIR CONFIGURATIONS

Algorithm	Compression Ratio against FP16	Vector Size	#Entry	Residual
QuiP#-4	25%	8	65536*	2
AQLM-3	18.75%	8	4096	2
GPTVQ-2	12.5%	4	256	1
CQ-4	25%	2	256	1
CQ-2	12.5%	4	256	1
Configs.		$2^{l_1, 2, \dots}$	$2^{l_1, 2, \dots}$	1, 2, ...

*QuiP# utilize a lattice-based codebook, though it has 65536 entries, it only need to look up from 256 of them every dequantization with bit operations.

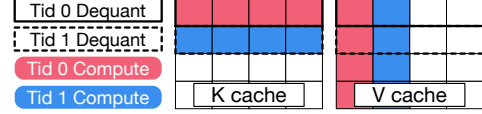


Fig. 6. Layout of dequantized data and required layout of following computation of KV cache in attention (decoding).

every (256, 256) tile of the weight matrix shares a codebook, while the task is spliced into $(\cdot, 128)$ tiles on weight matrix, and every two thread blocks access and load a same codebook.

Besides the increased off-chip global memory traffic, we also observe a significant rise in on-chip shared memory to register traffic in the **VQ-attn-SC** version, as shown in the last bar of Fig. 4 (right). Ideally, this traffic should remain the same to the **FP16-attn** version, given that the computation precision and the volume of data involved in the computation remain unchanged. The unusual Shared $\rightarrow$ Reg traffic stems from a mismatch between the layout of dequantized data and the layout required by the computation.

As illustrated in Fig. 6, one thread dequantizes a row of four elements at a time for the KV cache following the CQ-2 algorithm [69]. It then stores these four elements in thread-local registers. However, the computation requires a column-wise weighted accumulation on the V cache, and the four dequantized elements by the thread do not match the data needed for subsequent computations. Consequently, the dequantized data in local registers must be stored back into shared memory, allowing the correct threads to access them. Notice that as depicted in the figure, the K cache does not introduce such a shared memory round-trip since its row-wise accumulation process aligns with the dequantization process.

Takeaway 2 Integrating and fusing VQ algorithms into LLM’s kernels requires a careful coordination between the codebook load and the fused kernel’s compute dataflow.

C. Additional Complexity of VQ Diversity

Our above analysis primarily focuses on a specific VQ configuration for the FlashDecoding kernel. In fact, we have surveyed state-of-the-art methods of using VQs to accelerate LLMs and found considerable diversity, as listed in Tbl. II. These varied configurations add complexity when generating high-performance fused computation kernels. Moreover, different algorithms choose to train a codebook with different parts of tensor which further push up this complexity. For instance, QuiP# [56] can avoid duplicated Global $\rightarrow$ Shared traffic as it train one codebook with the entire weight tensor, yet it may increase bank conflicts and cause layout mismatches with its vector size 8. Conversely, CQ-4 [69] is able to reduce bank conflicts and layout issues with its vector size 2, but it may lead to significantly duplicated Global $\rightarrow$ Shared traffic since it train different codebooks with different channels.

On the other hand, there are various computations associated with VQ algorithms, such as **VQ-gemm** and **VQ-gemv** for weight-only quantization, and **VQ-attn** for KV cache quantization, as previously mentioned. The combination of

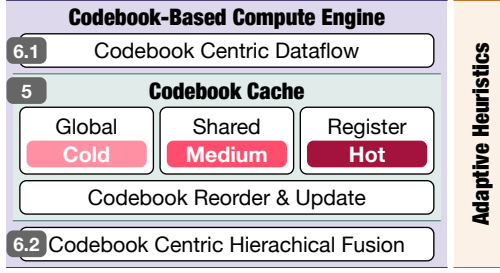


Fig. 7. VQ-LLM design overview.

VQ algorithm diversity and multiple subsequent computation patterns makes it impractical to manually craft efficient kernel implementations for each specific case.

Takeaway 3 An adaptive solution is necessary to achieve optimal performance across a variety of VQ algorithms and their subsequent computations.

IV. VQ-LLM OVERVIEW

From the analysis in the previous section, we identify three key challenges in utilizing VQ to accelerate LLM inference: i) efficient codebook entry access, ii) coordinated codebook loading and subsequent computation, and iii) significant diversity in VQ algorithms and subsequent computation patterns.

To address these challenges, we design and implement VQ-LLM, an automatic high-performance code generation framework in Fig. 7. We introduce a software abstraction called codebook cache to optimize codebook access efficiency and support the integration of VQ with various computations. The codebook cache **adaptively** stores different entries across the GPU’s memory hierarchy, including off-chip global memory, on-chip shared memory, and registers. It does so by leveraging the offline-profiled characteristics of codebook entry access, such as cold, medium, and hot.

The codebook cache also enables seamless integration with the subsequent computations. Centered around the codebook cache, we design an efficient computation engine that optimizes memory traffic during computations involving codebooks, incorporating two core techniques. The first technique, called codebook-centric dataflow, divides and parallelizes the original computation task in a way that minimizes the codebook switch overhead. It may split the reduction dimension of the original computation task, for which we **adaptively** determine the split factor to balance the global reduction. With this dataflow, we eliminate the excessive off-chip memory traffic caused by redundant codebook loads from different thread blocks in current VQ implementations.

The second technique employed by our compute engine, named codebook-centric hierarchical fusion, extends the default shared memory level fusion to support the additional register-level fusion. This mechanism leverages a GPU feature known as intra-warp data exchange [42] to rearrange the dequantized data into the required layout for subsequent computations directly in registers. And we **adaptively** decide where to conduct the fusion based on profiled exchanging

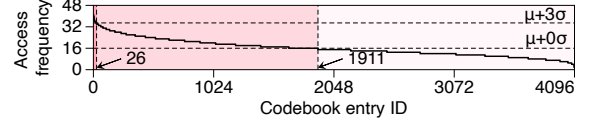


Fig. 8. Codebook entries access frequency of one thread block in VQ-GeMM kernel, with $VQ<8, 12, 2>$ (AQLM-3).

overhead and difference between layout of dequantized data and layout required by subsequent computation.

Our VQ-LLM framework comprises a set of CUDA templates that employ a codebook-centric dataflow and fusion scheme, along with a set of adaptive heuristics. These templates include both algorithm-specific and hardware-related parameters. To generate a specific VQ-augmented compute kernel, we supply the configuration of the algorithm and target GPU to the corresponding compute kernel template. VQ-LLM then automatically selects the optimal parameters based on the template specifications and heuristics.

V. CODEBOOK CACHE

We first present the design intuition of codebook cache, and then implementation details. Finally, we describe the user interface that can be utilized by subsequent computations.

A. Design Rationale

As Sec. III explains, naively placing the entire codebook in the shared memory results in suboptimal performance due to two issues: i) increased shared memory usage and ii) significant bank conflicts. To address these issues, we propose storing different entries at various memory levels based on their access frequencies. Specifically, we can store rarely accessed entries in off-chip global memory to conserve shared memory usage, and store the most frequently used entries in the thread local registers to eliminate bank conflicts.

We find that different entries in a codebook indeed demonstrate varying levels of ‘hotness’ in terms of access frequency. Fig. 8 illustrates such an example of AQLM-3, and results of other algorithms will be shown in Sec. VII. Over half of the codebook entries are accessed less frequently than the average, indicating that placing them in shared memory yields little benefit. There are 26 hot entries that are accessed more frequently than $\mu+3\sigma$ (mean plus three standard deviations), suggesting that they are more susceptible to inevitable bank conflicts. This observation forms the foundation of our codebook cache design, the details of which we introduce next.

B. Implementation

Typically, the implementation of a cache relies on tag array [59] or lookup table [36], which could incur additional latency and storage overhead. In our codebook cache implementation, we adopt a reorder-based static mapping mechanism that is extremely lightweight and configurable, which means there is also no complex eviction policy.

In our implementation, we first sort and reorder the codebook entries by their access frequency in the descending order.



Fig. 9. Entries hot and cold of different parts of tensor.

This is done at the profiling-based offline phase, which ensures that the index of the most frequent entry is 0, and the index of the least frequent entry is the maximum value. All the quantized data would use these new indices. Next, we establish two boundaries: n_{reg} and n_{shared} . We allocate the first n_{reg} entries to thread local registers and the subsequent entries up to n_{shared} in shared memory. We store any remaining entries in global memory. During runtime dequantization, addressing codebook entries involves simple index comparisons, we locate entries in registers if the index $< n_{reg}$, in shared memory if $n_{reg} \leq \text{index} < n_{shared}$, and in global memory if the index $\geq n_{shared}$.

In this implementation, we conduct frequency-based re-ordering at the tensor level, although different parts of a tensor might have different frequently accessed entries. Fig. 9 presents data to support our choice, where the y-axis represents different parts of the tensor (i.e., different thread blocks), and x-axis indicates the access frequencies of different codebook entries of a thread block. White color indicates frequently accessed entries, and the opposite for darker shades. We observe many vertical white lines, suggesting that these entries are consistently accessed across different tensor parts. This observation supports the rationale for globally determining the most frequently accessed entries.

Adaptivity. The shared memory and register resources of our codebook cache can be adjusted using two parameters: n_{reg} and n_{shared} . As mentioned in Sec. III, these resources are limited on GPUs, and excessive usage can decrease the occupancy of thread blocks. We employ a heuristic-based method that adapts their allocation to subsequent computations. Initially, we identify slack in the use of both resources. This concept is illustrated in Fig. 10, where we assign varying amounts of shared memory and registers to two computation kernels, highlighting the most performant configuration with a circle marker. Resource slack, depicted as the blue shaded area in Fig. 10, is a space of resource that we can occupy without hurting concurrency and GPU utilization. The existence of these slacks is due to the GPU's resource partitioning and scheduling [52], which we will not explore further due to space constraints. It is important to note that different computations exhibit varying slacks, which can also be derived by offline profiling. We determine n_{reg} and n_{shared} by dividing the available slacks by the size of a single codebook entry.

C. User Interface

We provide and explain the following APIs for users to utilize our codebook cache, henceforth abbreviated as CB.

```
CBcached, nreg,shared ← Load(CB, Slack)
Entry ← Access(CBcached, nreg,shared, CB, Index)
CB ← Switch(New CB Pointer)
```

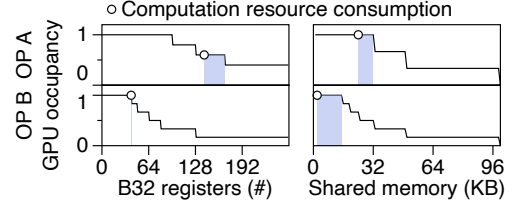


Fig. 10. Computation kernel resource consumption and corresponding occupancy of the hardware. The blue region is the resource slacks we can use without influencing the performance.

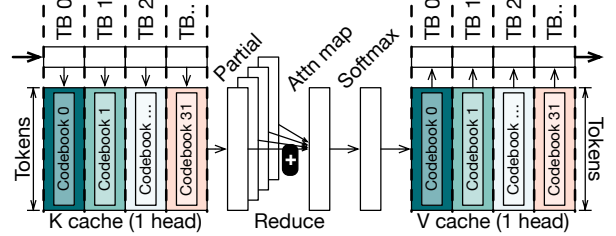


Fig. 11. Example of codebook-centric dataflow for attention (decode) computation following CQ configuration.

The first API is **Load**, which loads codebooks stored in global memory into the cache. It accepts the codebooks and memory slack, returning the codebooks cached across the memory hierarchy along with two access boundaries. The second API is **Access**, allowing users to access specific entries during the dequantization process. It accepts cached and global memory-stored codebooks along with indices to locate entries. It also uses two boundaries to determine where to locate entries. Additionally, while we configure these boundaries with preset heuristics, users can still overwrite them.

The last API is **Switch**, useful when algorithms train different codebooks for different parts of a tensor, as in GPTVQ-2 [57]. This API facilitates the switch to new codebooks based on the specific tensor section being processed by the user.

VI. CODEBOOK-BASED COMPUTE ENGINE

Based on the above codebook cache, we design an efficient compute engine to optimize the excessive codebook-related traffic when using VQ in the subsequent computation. We first introduce two core techniques employed by our computation engine: codebook-based dataflow and codebook-based hierarchical fusion. We then detail the combined usage of the entire computation engine along with the codebook cache.

A. Codebook Centric Dataflow

We start by explaining the intuition of our design. Subsequently, we detail our implementation.

1) *Design Rationale:* To fully leverage the parallel computation resources of GPUs, researchers employ tiling to divide and parallelize computation tasks [6], [43], [73]. Under the VQ scenario, naive parallelization introduces excessive traffic due to conflicts between the codebook switch axes and the task reduction axes, as discussed in Sec. III. We address this issue with a new codebook-centric dataflow illustrated in Fig. 11,

which employs the same settings as Fig. 5 in Sec. III. In this codebook-centric dataflow, we partition and parallelize the task across every four channels, i.e., every codebook, ensuring that each thread block only needs to load one codebook, thus eliminating any need for duplicated codebooks or switches. Instead of globally reducing the local softmax of different tokens as in FlashDecoding [10], we now require global accumulation of partial inner-products.

2) *Implementation*: We now formally define our design for the codebook-based dataflow. We first identify the axes where reduction occurs and where codebooks need to be switched, as indicated in Tbl. III. Subsequently, we split and parallelize the computation along the codebook switch axes. Finally, for those axes that traditionally perform temporal accumulation but are now parallelized (intersecting with the codebook switch axes and annotated with colors), we perform an explicit global reduction to ensure accurate results.

Adaptivity. To balance the overhead of global reduction in our dataflow, we utilize a split factor to control the extent of task parallelization along the codebook switch axes. A larger split factor results in fewer duplicated codebooks but necessitates more global reductions, and vice versa. With the objective of minimizing overhead, we adaptively determine the split factor based on the size of the tensor that needs reduction and the traffic associated with duplicated codebooks.

$$\begin{aligned} \text{Traffic}_{\text{Reduce}} &\leftarrow \text{Split Factor} \times \text{Output Size} \\ \text{Traffic}_{\text{Codebook}} &\leftarrow \frac{\text{Original Codebook Traffic}}{\text{Split Factor}} \end{aligned}$$

Since these two variables exhibit opposing trends with respect to the split factor, we can achieve a minimum by equating them according to the Mean Value Theorem [48].

B. Codebook Centric Hierarchical Fusion

Similarly, we begin with a concrete example to illustrate our new fusion scheme. Subsequently, we formally abstract the hierarchical fusion algorithm and detail our implementation.

1) *Design Rationale*: The baseline method described in Sec. III employs shared-memory-level fusion, which combines VQ dequantization and the subsequent computation kernel by transferring data through shared memory. It leads to excessive

TABLE III
REDUCE AND CODEBOOK SWITCH AXES OF COMPUTATIONS

GeMM GeMV	All axes	Reduce axes	Codebook switching axes
Weight	M,N,R	M,R	R : AQLM, QuiP# M,N : GPT-VQ
<i>R: Residual, M,N: M rows, N columns</i>			
Attention	All axes	Reduce axes	Codebook switching axes
K Cache V Cache	B,H,T,C B,H,T,C	C T	H,C : CQ H,C : CQ

B: Batch, H: Head, T: Token, C: Channel

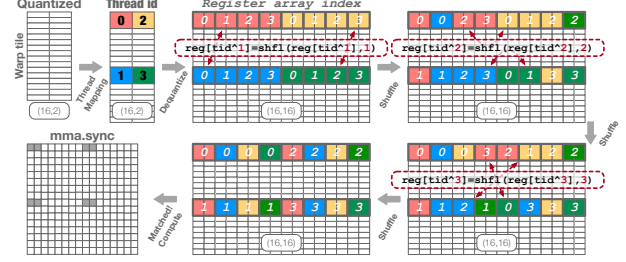


Fig. 12. Intra-warp data exchange based on shuffle API example, eight elements are dequantized one time per thread, while following computation requires one thread hold only two elements (mma instructions).

traffic between shared memory and registers, as previously explained. Alternatively, we utilize a modern GPU feature that facilitates register-level data exchange [42], effectively bypassing shared memory with following API:

$$\text{register} \leftarrow \text{shfl}_{\text{xor}}(\text{register}, \text{offset}) \quad (1)$$

This API exchanges the *reg* of the calling thread (*id_{src}*) with *reg* of the thread whose *id_{dst} ⊕ offset = id_{src}* in place ($\oplus: \text{xor}$). Note that this instruction is commonly used to enhance the efficiency of collective communication and result reduction [27], [72]. However, we are the first to apply it to accelerate VQ-compressed LLMs.

We illustrate the application of this API for register-level fusion through an example that fuses **VQ<8, ...>** with GeMM. In Fig. 12, the layout of the dequantized data is 8 (i.e., VQ vector size), while the layout required by the *mma* instruction is 2. We initially map the dequantization threads in a specialized manner, as depicted in the figure, to ensure that all data exchanges are confined to four threads, which we subsequently refer to as a mini-warp. Within this mini-warp, we execute three exchange (*shfl*) operations as follows:

- Tid 0.[1] ↔ Tid 1.[0], Tid 2.[3] ↔ Tid 3.[2]
- Tid 0.[2] ↔ Tid 2.[0], Tid 1.[3] ↔ Tid 3.[1]
- Tid 0.[3] ↔ Tid 3.[0], Tid 2.[1] ↔ Tid 1.[2]

Note that both the array index and thread ID can be represented using the *xor* operation. After these shuffle operations, the data held by each thread's register aligns precisely with the requirements of the *mma* computation instruction.

Thread Mapping. Our approach necessitates a specialized thread mapping within a warp for dequantization, as the naive sequential mapping requires a complex exchange pattern. Consider the sequential mapping with the *mma* instruction in Fig. 12, data[8,0:8] (blue color) is dequantized by thread 16 but is required by threads 0-3. However, the data held by threads 0-3 is not needed by thread 16 but rather by threads 0-7. This results in a complex data exchange path where ultimately all threads are implicated. Meanwhile, it requires additional registers as the exchange happens in place. To circumvent this, we predetermine the thread mapping offline, based on the layout of the dequantized data and the layout required by the computation, with details described as follows.

Algorithm 1 Intra-warp data exchange based on shuffling

Input: $data, iter, layout_{dequant}, compute$
Output: $data$

```
1: function THREAD_MAPPING( $data, layout_{dequant}, compute$ )
2:   for  $item \in data$  do
3:      $item.tid_{compute, dequant} \leftarrow GetTid(item, layout_{compute, dequant})$ 
4:    $mini\_warps \leftarrow []$ 
5:   for  $dequant\_thread \in warp$  do
6:      $mw \leftarrow [data.tid_{compute} \text{ for } data.tid_{dequant} = dequant\_thread]$ 
7:     if  $mw \notin mini\_warps$  then
8:        $mini\_warps[mw] \leftarrow []$ 
9:        $mini\_warps[mw].append(dequant\_thread)$ 
10:    for  $mw \in mini\_warps$  do
11:       $mini\_warps[mw][i] \leftarrow mw[i] // \text{Thread mapping we need}$ 
12: function REG_FUSION( $data, iter$ )
13:   for  $off$  in  $[1, iters]$  do // intersected 0 no shuffle needed
14:      $data[tid'off] \leftarrow shfl_{xor}(data[tid'off], off)$ 
15:   return  $reg$ 
```

2) *Implementation:* We outline our algorithm in Alg. 1. To determine the thread mapping, we first find the association between each element in terms of dequantization and computation (lines 2-3). Subsequently, for each thread, we identify all threads that require its dequantized data, grouping these threads into a mini-warp (lines 4-6). We then construct mini-warps for all threads (lines 7-9). In the previous example, threads 0, 1, 16, and 17 possess identical data $[0, 1, 2, 3]$ and thus form a mini-warp. Finally, we remap all threads by mini-warps (lines 10-11); for instance, we assign threads 2 and 3 to dequantize the data initially handled by threads 16 and 17. This process is executed offline to ensure proper thread mapping in runtime dequantization, enabling the implementation of register-level fusion via the shuffle API (lines 12-15).

Adaptivity. Clearly, a larger discrepancy between the dequantization layout and the required layout of the computation kernel increases the need for shuffling. Consequently, we propose conducting hierarchical fusion adapted to the vector size of the codebook entry. Profiling results indicate that the latency of shared memory access is nearly five times that of register access combined with shuffling. Therefore, for quantized tensors requiring fewer than five shuffle operations, we implement register-level fusion. For other tensors, we maintain the conventional shared memory-level fusion.

C. Overall Workflow

Our compute engine adopts a template-based design in Alg. 2 to generate final fused kernels. First at the offline phase, based on the VQ configuration and targeted computation, we determine shared/register budgets, split factors, required number of shuffles, and the corresponding thread mapping for our proposed optimizations (lines 2-8).

Subsequently, we launch the codebook-centric dataflow computation (line 9) via the **Parallel_For** function that binds following sub-tasks to parallel thread blocks. Its two parameters represent the task splitting axes and the split factor, respectively. Within each parallelized task, we first load the codebook into the codebook cache (lines 10-12), followed by dequantization using the provided APIs in Sec. V (lines

13-14). Notice now threads are mapped to quantized data following **Thread Mapping** determined offline, for minimum shuffle if applicable. After dequantization, we perform codebook-centric hierarchical fusion (lines 15-18) using the **Reg_Fusion** and **Shared_Fusion** function. Both functions accept dequantized data, with the former requiring a counter $n_{shuffle}$ to indicate the number of required shuffle operations and latter requiring the source-destination layout to initialize correct shared memory accesses. Once the data is in the proper layout, we proceed with computation (lines 19-20). Finally, we perform a global reduction if necessary (line 21) via the **Reduce** function, where the first parameter specifies the partial result to be reduced and the second determines the axes along which the global reduction is conducted.

Algorithm 2 Complete VQ-aware computation template

Input: $quantized, codebook, compute_op$
Output: $output$

```
1: function KERNEL_TEMPLATE
2:    $All, Reduce \leftarrow compute\_op.all\_axes, reduce\_axes$ 
3:    $layout_{src, dst} \leftarrow codebook.vector\_size, compute\_op.required\_size$ 
4:    $Budget \leftarrow \text{Free shared and reg to preserve occupancy}$ 
5:    $factor \leftarrow \text{Value to make } Traffic_{Reduce} = Traffic_{Codebook}$ 
6:    $n_{shuffle} \leftarrow layout_{src} / layout_{dst}$ 
7:   if  $n_{shuffle} \leq thres_{shuffle} (= 5)$  then
8:     Thread_Mapping( $compute\_op.warp\_tile, layout_{src, dst}$ )
9:   Parallel_For( $codebook.switch\_axes, factor$ )
10:    if required by algorithm then
11:       $CB \leftarrow \text{Switch}(\text{New codebook ptr})$ 
12:       $CB_{cached}, boundry \leftarrow \text{Load}(CB, Budget)$ 
13:      for  $id$  in  $quantized\_data$  do
14:         $data \leftarrow \text{Access}(CB_{cached}, boundry, CB, id)$ 
15:      if  $n_{shuffle} \leq thres_{shuffle}$  then
16:         $data \leftarrow \text{Reg\_Fusion}(data, n_{shuffle})$ 
17:      Else
18:         $data \leftarrow \text{Shared\_Fusion}(data, layout_{src, dst})$ 
19:      for  $temporal\_iteration$  on  $All - codebook.switch\_axes$  do
20:         $partial \leftarrow compute\_op(data, temporal\_iteration)$ 
21:       $output \leftarrow \text{Reduce}(partial, Reduce \cap codebook.switch\_axes)$ 
22:   Return  $output$ 
```

VII. EVALUATION

In this section, we evaluate the effectiveness of proposed optimizations in VQ-LLM through comprehensive experiments. We first present overall speedup results for various VQ-based computation kernels over existing approaches. Then, we provide a detailed breakdown analysis of the proposed optimizations. Next, we compare our work with FP16 kernels and several element-wise quantization works to show its viability for accelerating LLMs. Finally, we performed a comprehensive end-to-end evaluation, analyzing both the overall speedup and accuracy across various GPUs.

A. Experimental Setup

In this study, we conduct a comprehensive evaluation at both the individual kernel and end-to-end model levels. The evaluations were performed on an NVIDIA RTX 4090 24GB GPU [44]. For the end-to-end evaluation, we included a Tesla A40 GPU [39] to explore the potential of VQ-LLM with lower bandwidth. The evaluated computation kernels

TABLE IV
BREAK DOWN ANALYSIS CONFIGURATION

ID	Optimization Category	Description
GC	No	Naive implementation
SC	Greedy	Cache all entries in shared memory
O1 O2	Hierarchical Buffer	+ Shared memory level caching (medium entries) + Register level caching (hot entries)
O3 O4	Compute Engine	+ Codebook centric dataflow + Codebook centric hierarchical fusion

include various VQ-augmented GeMM, GeMV and FlashDecoding [10]. The evaluated VQ configurations are listed in Tbl. II, including QuiP#-4, AQLM-3, GPTVQ-2 and CQ-2/4, the suffix number represent the equivalent bit-width. The first two kernels adopt weight quantization and the last one adopts KV cache quantization. For the kernel-level evaluation, we set the shape for these kernels following the Llama-7B and Llama-65B [55] models. These kernels run on a single GPU, while large model serving like Llama-65B typically uses multiple GPUs with Tensor Parallel (TP) strategy [35], [47], [71]. The required adjustments to our framework include final results gathering for Attention and partial results concatenation/reduction for GeMM/GeMV [38]. These are usually conducted via communication library like NCCL [45], and we identify this distributed scenario an orthogonal topic and leave it to the future work.

Tbl. IV lists various baselines and VQ-LLM optimizations used in our evaluation. For the baselines, we use **GC** and **SC** method explained in Sec. III that stores the codebook in global memory and shared memory, respectively. For the results, we report the latency reduction against **GC**. We also decompose the optimizations used in VQ-LLM into four levels (**O1-O4**), with each explained in Tbl. IV. We also compare VQ-LLM with SOTA element-wise quantization methods under the same equivalent 4-bit width, including AWQ [30] for GeMM/GeMV and QoQ for Attention [31], all integrated into qServe [31]. For FP16, we use cutlass [43] and flash-attn [8].

In practice, LLM inference involves various operators beyond GeMM/GeMV and Attention, such as RMSNorm [66], SiLU [14], and RoPE [53], etc. Therefore, it is crucial to evaluate the end-to-end speedup that accounts for all operators. For the end-to-end evaluation, we set a batch size of 16 and a sequence length of 1024, measuring the total latency for generating 256 tokens. We also assess accuracy using the arc-challenge task [5], applying the QuiP#-4 and CQ-4 algorithms for quantizing the weights and KV-Cache, respectively. To obtain the final accuracy results, we integrate these algorithms into the LMEval framework [16].

B. Overall Speedup

As shown in Fig. 13, VQ-LLM reduces the latency by an average of 46.13% (53.73% at most), corresponding to a speedup of $1.9\times$ ($2.2\times$) (BS x indicates the batch size of x).

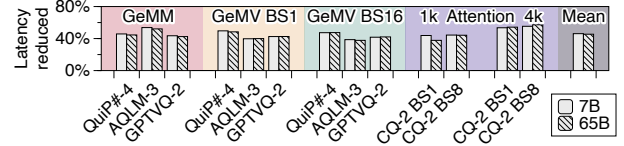


Fig. 13. Overall latency reduction of best perform version against un-optimized version for various VQ configurations.

For Attention (Decode), 1k and 4k means sequence length of 1024 and 4096, respectively.

Although VQ-LLM achieves significant speedup values for both GeMM and GeMV kernels, we observe a counter-intuitive discrepancy that our optimizations achieve a relatively high speedup value for GeMM kernels compared to GeMV kernels. In other words, the quality of VQ algorithm integration is more critical to the compute-bound kernels (e.g., GeMM) than to the memory-bound kernels (e.g., GeMV). The reason is the former benefit less from reduced memory footprint while suffer more from extra operation (dequantization) [60], leading to significant performance degradation of unoptimized implementation. Meanwhile, we also observe an opposite trend for AQLM-3 between GeMM and GeMV. This AQ configuration has an unaligned 12-bit storage format, which necessitates additional unpacking and decoding logic and requires a more careful optimization for the integration.

We observe that our speedup values for GeMV kernels remain consistent regardless of batch size, whereas they increase with batch size for attention kernels. This is because different input samples share the same weight tensor but have distinct KV caches. Since the GeMV kernel corresponds to weight quantization and the attention kernel to KV cache quantization, the former only requires loading the VQ-compressed weight tensor once, while the latter loads VQ-compressed KV cache tensors multiple times. Consequently, our optimizations are more effective for the attention kernel with large batch sizes.

Moreover, Llama-65B achieves almost identical speedup to Llama-7B, except in the Attention (Decode) scenario with a 1k sequence length and a single batch. This identical speedup occurs because the operators in the larger model can be trivially assembled using those from the smaller ones. We can readily double the launched thread blocks when we double the hidden dimension, demonstrating the good scalability of our optimizations. The sole exception arises because, in Llama-7B, the baseline cannot fully utilize the hardware due to an insufficient number of thread blocks for a 1k sequence length single batch. In contrast, for Llama-65B, the baseline fully occupies the hardware, resulting in better performance and reducing the relative speedup of our system.

C. Speedup Breakdown

We first analyze the speedup breakdown of GeMM and GeMV, as depicted in Fig. 14. Tbl. V enumerates several factors that influence optimization effects, facilitating our analysis. For QuiP#-4, **SC** and **O1** perform identically due to the small size of its codebook (i.e., 2 KB in Tbl. V).

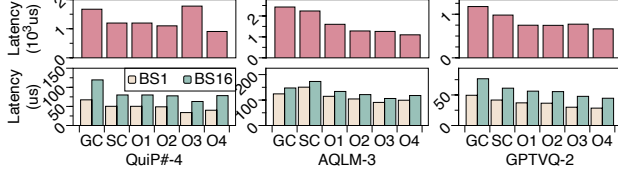


Fig. 14. Breakdown of optimizations for GeMM (upper) and GeMV (lower).

TABLE V
FACTORS THAT INFLUENCE THE EFFECT OF OPTIMIZATIONS

Item	QuiP#-4	AQLM-3	GPTVQ-2	CQ-2
Codebook/block	2 KB	128 KB	32 KB	64 KB
#Entry freq $> \mu+3\sigma$	1-3	15-30	<1	<1
Output size/block		32 KB/ <1 KB*		1-4 KB
#Shuffle	3/7*	3/7*	1/3	3

*GeMM/GeMV

AQLM-3 and GPTVQ-2 exhibit noticeable improvements, attributed to their larger codebooks. Additionally, for GeMV, **SC** has a significantly negative impact on AQLM-3, due to its large codebook (i.e., 128 KB in Tbl. V), which restricts the parallelization of memory-bound computations.

O2 delivers the most improvement in AQLM-3; we find that frequencies of 15-30 entries exceed $\mu+3\sigma$, and **O2**'s register-level caching optimization effectively reduces bank conflicts when accessing these entries. Conversely, the remaining two configurations QuiP#-4 and GPTVQ-2 exhibit far fewer entries exceeding $\mu+3\sigma$, indicating the less optimization opportunity of register-level caching and hence marginal improvements.

O3 affects GeMM and GeMV differently. In GeMM, **O3** introduces negative effects due to a large output size. Furthermore, multiple residuals in QuiP#-4 configuration lead to redundant computations for **O3**, causing significantly increased latency in GeMM. In contrast, for AQLM-3, its misaligned 12-bit indices result in costly unpacking and decoding. It leads to low compute pipeline utilization, and hence is more tolerant to redundant computations. In GeMV, the output size is much smaller and the computation is lighter compared to GeMM. The smaller output size results in minimal global reduction overhead, and the lighter computation introduces less computational overhead than in GeMM. These factors make **O3** more advantageous for GeMV.

O4 significantly enhances GeMM's performance. This improvement primarily stems from GeMM's utilization of `mma` instructions, which require a layout of 2 and can be satisfied through one to three shuffling instructions. Additionally, **O4** conserves a substantial amount of shared memory, which is crucial as GeMM typically consumes a large shared memory, thus yielding a significant positive impact. Conversely, GeMV requires element-wise reduction, resulting in QuiP#-4 and AQLM-3, with a vector size of 8, requiring a greater number of shuffling instructions. This leads to a slowdown in these configurations. However, for GPTVQ-2 with a vector size of 4, a slight improvement is still observed. Furthermore, since GeMV typically uses minimal shared memory, savings in this

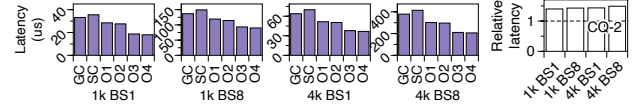


Fig. 15. (left) Breakdown of optimizations of CQ-2 for Attention (Decode). (right) Relative latency of CQ-4 against CQ-2.

area have a lesser impact on performance.

For Attention (decode), VQ-LLM achieves similar improvements with various sequence lengths and batches. **SC** significantly reduces performance due to CQ's large codebook, necessitating the use of **O1** for achieving high performance. **O2** offers only a slight improvement because few entries are accessed very frequently, mirroring situations in QuiP#-4 and GPTVQ-2. **O3** significantly enhances performance by eliminating considerable duplicated traffic in the original computation dataflow. **O4** provides a minor improvement, for reasons similar to those for **O4** in GeMV. Additionally, we illustrate the latency of CQ-4 relative to CQ-2 in the right part of Fig. 15. Our proposed optimizations achieve a similar speedup to CQ-2, so we omit the detailed results to save space.

D. FP16 and Element-wise Quantization Comparison

We now compare the latency of our optimized VQ kernels against FP16 and element-wise quantization works. Under the same equivalent bit width, the latency of kernels with the element-wise quantization is the theoretical upper bound of VQ kernels if using the same computation dataflow. As such, this comparison further verifies the effectiveness of our work.

As shown in Fig. 16, at 4-bit encoding, our work achieves latencies comparable to ($1.01\times$ for Attention (Decode)), or even lower than ($0.88\times/0.96\times$ for GeMV/GeMM), those of AWQ [30] and QoQ [31]. This reduction in latency likely results from our co-designed computational dataflow. These results suggest that our implementation is as viable as AWQ and QoQ, and therefore comparable to qServe [31]. Moreover, VQ kernels can deliver better accuracies at the same bit-width. The open-source implementations of QuiP# [56] and AQLM [12] are impractical for real-world applications, exhibiting $2.83\times$ to $114.4\times$ latencies. Our work successfully translates theoretical algorithmic improvements into practical applications.

We would like to explain that in Fig. 16, while both our approach and element-wise quantization methods outperform the cutlass-FP16 baseline in GeMV and Attention kernels, both underperform relative to the cutlass-FP16 baseline in GeMM kernels. This underperformance is due to the complex tiling

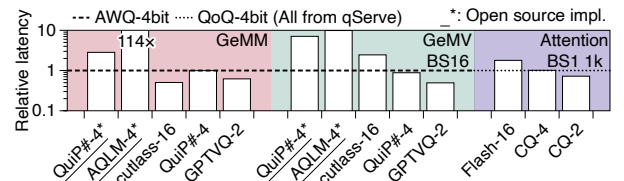


Fig. 16. Latency comparing to element-wise quantization works.

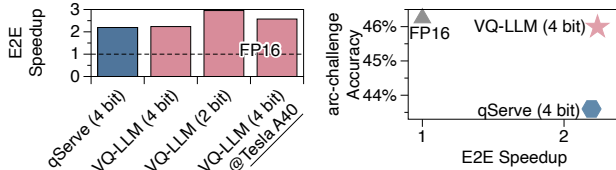


Fig. 17. (left) Overall speedup against FP16 and (right) accuracy of arc-challenge of SOTA element-wise quantization (qServe) and VQ-LLM.

strategy employed by cutlass-FP16 GeMM, which could incorporate our method. However, we do not pursue this integration for two reasons. First, accelerating individual GeMM kernels offers minimal overall speedup for LLM inference, as these kernels are used in the prefilling stage (Sec. II-B). The decoding stage, which dominates LLM inference execution time, has a greater impact on performance [64], as confirmed by our end-to-end evaluation results in the next subsection. Second, modifying the cutlass code requires significant engineering effort due to its intricate, template-based kernel design [22], [76]. Therefore, we leave this integration for future work.

E. End-to-End (E2E) Result

We present the end-to-end LLM inference improvements of various quantization methods compared to the FP16 baseline in Fig. 17 (left). In the equivalent 4-bit setting, VQ-LLM achieves a speedup comparable to the state-of-the-art element-wise quantization method, qServe [31], with both providing approximately a $2.2\times$ improvement over the FP16 baseline. Additionally, VQ-LLM surpasses qServe in accuracy by about 2.5% on the arc-challenge task [5], as shown in Fig. 17 (right). This result demonstrates VQ-LLM's effectiveness in accelerating LLM inference. The RMSNorm, SiLU, and RoPE operators together account for roughly 10% and 20% of total latency in the FP16 and 4-bit quantized versions, respectively. We also observe a greater speedup with a 2-bit compression ratio, further highlighting the potential of VQ, as previous research suggests that 2-bit quantization can maintain practical accuracy. Additionally, we evaluate the performance of VQ-LLM in a 4-bit setting on the Tesla A40 GPU, which provides 67% of the memory bandwidth of the RTX 4090 [39]. Interestingly, the Tesla A40 demonstrates a greater speedup than the RTX 4090, suggesting that VQ-LLM is more effective in bandwidth-constrained environments. In summary, VQ-LLM offers improved accuracy with comparable latency to element-wise quantization, and vice versa. In terms of memory usage, the FP16 baseline consumes over 22 GB, whereas qServe-4 and VQ-LLM-4 use less than 6 GB of GPU memory, aligning closely with theoretical estimates [65].

F. Additional Discussion

Different Types of Attention. The aforementioned details about Attention (Decode) are based on using Flash Decoding [10] as our baseline dataflow. We also evaluate the speedup of our work against various attention baselines, including Flash

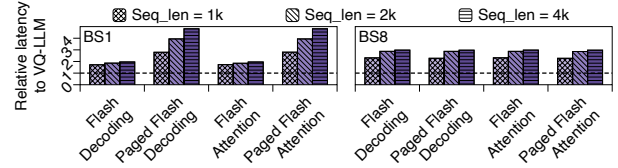


Fig. 18. Relative latency of various attention baselines against our best perform implementation of CQ-4.

Attention, Paged Flash Attention and Paged Flash Decoding [7]–[9], [50]. As illustrated in Fig. 18, our work surpasses all these baselines, primarily due to a significantly reduced KV cache memory footprint enabled by CQ-4. We achieved a 66.4% latency reduction compared to the best-performing FP16 baseline, with a 75% reduction in memory footprint, under the conditions of eight batches and a sequence length of 4096. This indicates an effective transfer from theoretical benefit to practical application. Additionally, our work scales effectively with increases in sequence length and batch size.

Quantization Overhead. For weight compression, no runtime quantization overhead is introduced. In KV cache compression, the runtime overhead of on-the-fly quantization for the new key and value of a new token in the decode phase is negligible ($<1 \mu s$). During the prefill phase, quantizing the keys and values of all prompt tokens introduces less than a 10% overhead compared to linear projections. However, the subsequent computation does not immediately require the quantized KV cache, rendering these overheads negligible.

VIII. CONCLUSIONS

In this work, we proposed VQ-LLM, an optimized code generation framework for vector quantization (VQ), consisting of codebook cache and codebook based compute engine. With which we achieve 46.13% latency reduction on average over unoptimized version and up-to 99% over open source implementations. For codebook cache, we propose a hierarchical placement strategy to preserve hardware utilization and reduce bank conflict. For compute engine, we propose codebook centric dataflow and fusion scheme to reduce excessive off-chip and on-chip traffic. All proposed optimizations are configured adaptively via several heuristics. Finally we demonstrate effectiveness and viability of VQ-LLM comparing to un-optimized implementations and element-wise quantization works.

IX. ACKNOWLEDGEMENTS

This work was supported by the National Key R&D Program of China under Grant 2022YFB4501400, the National Natural Science Foundation of China (NSFC) grant (62222210, U21B2017 and 62072297). This work was also supported by Shanghai Qi Zhi Institute Innovation Program SQZ202316. We would like to thank the anonymous reviewers for their constructive feedback and comments to improve this work. Special thanks to Vega Jiang for continuous help and support. Any opinions, findings and conclusions in this paper are those of the authors only and do not necessarily reflect the views of our sponsors.

REFERENCES

- [1] “Ieee standard for floating-point arithmetic,” *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.
- [2] E. Agrell and B. Allen, “On the best lattice quantizers,” *IEEE Trans. Inf. Theory*, vol. 69, no. 12, pp. 7650–7658, 2023.
- [3] AMD, “Amd cdna architecture: The all-new amd gpu architecture for the modern era of hpc & ai,” <https://www.amd.com/content/dam/amd/en/documents/instinct-business-docs/white-papers/amd-cdna-white-paper.pdf>, 2020.
- [4] A. Babenko and V. S. Lempitsky, “Additive quantization for extreme vector compression,” in *2014 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, OH, USA, June 23-28, 2014*. IEEE Computer Society, 2014, pp. 931–938.
- [5] S. Bhakthavatsalam, D. Khashabi, T. Khot, B. D. Mishra, K. Richardson, A. Sabharwal, C. Schoenick, O. Tafjord, and P. Clark, “Think you have solved direct-answer question answering? try arc-da, the direct-answer AI2 reasoning challenge,” *CoRR*, vol. abs/2102.03315, 2021.
- [6] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Q. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, “TVM: an automated end-to-end optimizing compiler for deep learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*. USENIX Association, 2018, pp. 578–594. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/chen>
- [7] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” *CoRR*, vol. abs/2307.08691, 2023.
- [8] —, “flash-attention,” 2024. [Online]. Available: <https://github.com/Dao-AILab/flash-attention>
- [9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- [10] T. Dao, D. Haziza, F. Massa, and G. Sizov, “Flash-decoding for long-context inference,” 2023. [Online]. Available: <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>
- [11] M. DouzeIR, “Faiss the index factory,” 2024. [Online]. Available: <https://github.com/facebookresearch/faiss/wiki/The-index-factory>
- [12] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, “Extreme compression of large language models via additive quantization,” *CoRR*, vol. abs/2401.06118, 2024.
- [13] —, “Official pytorch repository for extreme compression of large language models via additive quantization,” 2024. [Online]. Available: <https://github.com/vahe1994/AQLM>
- [14] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” *Neural Networks*, vol. 107, pp. 3–11, 2018.
- [15] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: accurate post-training quantization for generative pre-trained transformers,” *CoRR*, vol. abs/2210.17323, 2022.
- [16] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h, H. Li, K. McDonnell, N. Muenighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, “A framework for few-shot language model evaluation,” 07 2024.
- [17] T. Ge, K. He, Q. Ke, and J. Sun, “Optimized product quantization,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 4, pp. 744–755, 2014.
- [18] C. Guo, Y. Qiu, J. Leng, X. Gao, C. Zhang, Y. Liu, F. Yang, Y. Zhu, and M. Guo, “Squant: On-the-fly data-free quantization via diagonal hessian approximation,” in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [19] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*. ACM, 2023, pp. 3:1–3:15.
- [20] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “ANT: exploiting adaptive numerical data type for low-bit deep neural network quantization,” in *55th IEEE/ACM International Symposium on Microarchitecture, MICRO 2022, Chicago, IL, USA, October 1-5, 2022*. IEEE, 2022, pp. 1414–1433.
- [21] C. Guo, R. Zhang, J. Xu, J. Leng, Z. Liu, Z. Huang, M. Guo, H. Wu, S. Zhao, J. Zhao, and K. Zhang, “Gmlake: Efficient and transparent GPU memory defragmentation for large-scale DNN training with virtual memory stitching,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*. ACM, 2024, pp. 450–466.
- [22] B. Hagedorn, B. Fan, H. Chen, C. Cecka, M. Garland, and V. Grover, “Graphene: An IR for optimized tensor computations on gpus,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*. ACM, 2023, pp. 302–313.
- [23] P. Haghani, S. Michel, P. Cudré-Mauroux, and K. Aberer, “LSH at large - distributed KNN search in high dimensions,” in *11th International Workshop on the Web and Databases, WebDB 2008, Vancouver, BC, Canada, June 13, 2008*, 2008.
- [24] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “Kvquant: Towards 10 million context length LLM inference with KV cache quantization,” *CoRR*, vol. abs/2401.18079, 2024.
- [25] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.
- [26] B. Kövesi, J. Boucher, and S. Saoudi, “Stochastic k-means algorithm for vector quantization,” *Pattern Recognit. Lett.*, vol. 22, no. 6/7, pp. 603–610, 2001.
- [27] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “vllm: Easy, fast, and cheap llm serving with pagedattention,” 2024. [Online]. Available: <https://blog.vllm.ai/2023/06/20/vllm.html>
- [28] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*. ACM, 2023, pp. 611–626.
- [29] Y. Lee, H. Choi, S. Min, H. Lee, S. Beak, D. Jeong, J. W. Lee, and T. J. Ham, “ANNA: specialized architecture for approximate nearest neighbor search,” in *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022, Seoul, South Korea, April 2-6, 2022*. IEEE, 2022, pp. 169–183.
- [30] J. Lin, J. Tang, H. Tang, S. Yang, W. Chen, W. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*. mlsys.org, 2024.
- [31] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, “Qserve: W4A8KV4 quantization and system co-design for efficient LLM serving,” *CoRR*, vol. abs/2405.04532, 2024.
- [32] S. Liu, H. Lu, and J. Shao, “Improved residual vector quantization for high-dimensional approximate nearest neighbor search,” *CoRR*, vol. abs/1509.05195, 2015.
- [33] Z. Liu, J. Leng, Z. Zhang, Q. Chen, C. Li, and M. Guo, “VELTAIR: towards high-performance multi-tenant deep learning services via adaptive compilation and scheduling,” in *ASPLOS ’22: 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, 28 February 2022 - 4 March 2022*. ACM, 2022, pp. 388–401.
- [34] Z. Liu, W. Ni, J. Leng, Y. Feng, C. Guo, Q. Chen, C. Li, M. Guo, and Y. Zhu, “JUNO: optimizing high-dimensional approximate nearest neighbour search with sparsity-aware algorithm and ray-tracing core mapping,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*. ACM, 2024, pp. 549–565.
- [35] G. Lu, R. Chen, Y. Wang, Y. Zhou, R. Zhang, Z. Hu, Y. Miao, Z. Cai, L. Li, J. Leng, and M. Guo, “Distim: A performance model of large-scale hybrid distributed DNN training,” in *Proceedings of the 20th ACM International Conference on Computing Frontiers, CF 2023, Bologna, Italy, May 9-11, 2023*. ACM, 2023, pp. 112–122.
- [36] Z. Ma, Y. Tan, H. Jiang, Z. Yan, D. Liu, X. Chen, Q. Zhuge, E. H. Sha, and C. Wang, “Unified-tp: A unified TLB and page table cache structure for efficient address translation,” in *38th IEEE International Conference on Computer Design, ICCD 2020, Hartford, CT, USA, October 18-21, 2020*. IEEE, 2020, pp. 255–262.

- [37] E. Mata, S. Bandeira, P. S. G. de Mattos Neto, W. T. A. Lopes, and F. Madeiro, "Accelerating families of Fuzzy K -Means algorithms for vector quantization codebook design," *Sensors*, vol. 16, no. 11, 2016.
- [38] D. Narayanan, M. Shoneyi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on GPU clusters using megatron-lm," in *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2021, St. Louis, Missouri, USA, November 14-19, 2021*. ACM, 2021, p. 58.
- [39] NVIDIA, "Nvidia ampere ga102 gpu architecture," <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>, 2021.
- [40] —, "Nvidia h100 tensor core gpu architecture," <https://www.advancedclustering.com/wp-content/uploads/2022/03/gtc22-whitepaper-hopper.pdf>, 2022.
- [41] —, "Cuda c++ programming guide," 2024. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- [42] —, "Cuda c++ programming guide: Shared memory," 2024. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#shared-memory-5-x>
- [43] —, "Cuda templates for linear algebra subroutines," 2024. [Online]. Available: <https://github.com/NVIDIA/cutlass>
- [44] —, "Nvidia ada craft the engineering marvel of the rtx 4090," 2024. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/ada-lovelace-architecture/nvidia-ada-gpu-craft.pdf>
- [45] —, "Nvidia collective communications library (nccl)," 2024. [Online]. Available: <https://developer.nvidia.com/nccl>
- [46] OpenAI, "GPT-4 technical report," *CoRR*, vol. abs/2303.08774, 2023.
- [47] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: memory optimizations toward training trillion parameter models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*. IEEE/ACM, 2020, p. 20.
- [48] K. A. Ross, *Elementary Analysis: The Theory of Calculus*. Springer New York, NY, 2013.
- [49] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi *et al.*, "Microscaling data formats for deep learning," *CoRR*, vol. abs/2310.10537, 2023.
- [50] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," *CoRR*, vol. abs/2407.08608, 2024.
- [51] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "OmniQuant: Omnidirectionally calibrated quantization for large language models," *CoRR*, vol. abs/2308.13137, 2023.
- [52] G. Shobaki, A. Kerbow, and S. Mekhanoshin, "Optimizing occupancy and ILP on the GPU using a combinatorial approach," in *CGO '20: 18th ACM/IEEE International Symposium on Code Generation and Optimization, San Diego, CA, USA, February, 2020*. ACM, 2020.
- [53] J. Su, M. H. M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.
- [54] M. Thread, "Mtt s4000: Empower large model ai with no limits," <https://en.mthreads.com/product/S4000>, 2024.
- [55] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M. Lachaux *et al.*, "Llama: Open and efficient foundation language models," *CoRR*, vol. abs/2302.13971, 2023.
- [56] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. D. Sa, "Quip#: Even better LLM quantization with hadamard incoherence and lattice codebooks," *CoRR*, vol. abs/2402.04396, 2024.
- [57] M. van Baalen, A. Kuzmin, M. Nagel, P. Couperus, C. Bastoul, E. Mahurin, T. Blankevoort, and P. N. Whatmough, "GPTVQ: the blessing of dimensionality for LLM quantization," *CoRR*, vol. abs/2402.15319, 2024.
- [58] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA, 2017*, pp. 5998–6008.
- [59] H. Wang, T. Sun, and Q. Yang, "CAT - caching address tags: A technique for reducing area cost of on-chip caches," in *Proceedings of the 22nd Annual International Symposium on Computer Architecture, ISCA '95, Santa Margherita Ligure, Italy, June 22-24, 1995*. ACM, 1995.
- [60] S. Williams, A. Waterman, and D. A. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [61] C. Wolters, X. Yang, U. Schlichtmann, and T. Suzumura, "Memory is all you need: An overview of compute-in-memory architectures for accelerating large language model inference," *CoRR*, vol. abs/2406.08413, 2024.
- [62] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023.
- [63] D. Yang, S. Liu, R. Huang, J. Tian, C. Weng, and Y. Zou, "Hifi-codes: Group-residual vector quantization for high fidelity audio codec," *CoRR*, vol. abs/2305.02765, 2023.
- [64] L. Yu and J. Li, "Stateful large language model serving with pensieve," *CoRR*, vol. abs/2312.05516, 2023.
- [65] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, Y. Yan, B. Chen, G. Sun, and K. Keutzer, "Llm inference unveiled: Survey and roofline model insights," 2024.
- [66] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada, 2019*.
- [67] H. Zhang, X. Ji, Y. Chen, F. Fu, X. Miao, X. Nie, W. Chen, and B. Cui, "Pqcache: Product quantization-based kv-cache for long context llm inference," *CoRR*, vol. abs/2407.12820, 2024.
- [68] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. T. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "OPT: open pre-trained transformer language models," *CoRR*, vol. abs/2205.01068, 2022.
- [69] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, "KV cache is 1 bit per channel: Efficient large language model inference with coupled quantization," *CoRR*, vol. abs/2405.03917, 2024.
- [70] H. Zhao, W. Cui, Q. Chen, Y. Zhang, Y. Lu, C. Li, J. Leng, and M. Guo, "Tacker: Tensor-cuda core kernel fusion for improving the GPU utilization while ensuring qos," in *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022, Seoul, South Korea, April 2-6, 2022*. IEEE, 2022, pp. 800–813.
- [71] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing, J. E. Gonzalez, and I. Stoica, "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," in *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*. USENIX Association, 2022, pp. 559–578. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>
- [72] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. W. Barrett, and Y. Sheng, "Efficiently programming large language models using sglang," *CoRR*, vol. abs/2312.07104, 2023.
- [73] S. Zheng, S. Chen, S. Gao, L. Jia, G. Sun, R. Wang, and Y. Liang, "Tileflow: A framework for modeling fusion dataflow via tree-based analysis," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2023, Toronto, ON, Canada, 28 October 2023 - 1 November 2023*. ACM, 2023, pp. 1271–1288.
- [74] S. Zheng, S. Chen, P. Song, R. Chen, X. Li, S. Yan, D. Lin, J. Leng, and Y. Liang, "Chimera: An analytical optimizing framework for effective compute-intensive operators fusion," in *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*. IEEE, 2023.
- [75] Y. Zhou, J. Leng, Y. Song, S. Lu, M. Wang, C. Li, M. Guo, W. Shen, Y. Li, W. Lin, X. Liu, and H. Wu, "ugrapher: High-performance graph operator computation via unified abstraction for graph neural networks," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*. ACM, 2023, pp. 878–891.
- [76] Y. Zhou, M. Yang, C. Guo, J. Leng, Y. Liang, Q. Chen, M. Guo, and Y. Zhu, "Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators," in *IEEE International Symposium on Workload Characterization, IISWC 2021, Storrs, CT, USA, November 7-9, 2021*. IEEE, 2021, pp. 214–225.